



Production Information Systems

Topic 2 : DataBase Modeling and Design

Complement to CHAPTER 3 Transforming E/R diagrams into Relations

ISE Department
Year 4/ Level 7

Instructor : Dr. Jomana Bashatah

Normalization produces a set of well-structured relations that contains all of the data mentioned in system inputs and outputs developed in human interface design. Because these specific information requirements may not represent all future information needs, the E-R diagram you developed in conceptual data modeling is another source of insight into possible data requirements for a new application system. To compare the conceptual data model and the normalized relations developed so far, your E-R diagram must be transformed into relational notation, normalized, and then merged with the existing normalized relations.

Transforming an E-R diagram into normalized relations and then merging all the relations into one final, consolidated set of relations can be accomplished in four steps :

1. Represent entities: Each **entity** type in the E-R diagram becomes a **relation**. The identifier of the entity type becomes the primary key of the relation, and other attributes of the entity type become nonprimary key attributes of the relation.

2. Represent relationships: Each relationship in an E-R diagram must be represented in the relational database design. How we represent a relationship **depends on its nature**. For example, in some cases we represent a relationship by making the primary key of one relation a **foreign key** of another relation. In other cases, we create a **separate relation** to represent a relationship.

3. Normalize the relations: The relations created in steps 1 and 2 may have unnecessary redundancy. So we need to **normalize** these relations to make them well structured.

4. Merge the relations: So far in database design we have created various relations from both a bottom-up normalization of user views and from transforming one or more E-R diagrams into sets of relations. Across these different sets of relations, there may be redundant relations that must be merged and renormalized to remove the redundancy.

1. *Represent entities*

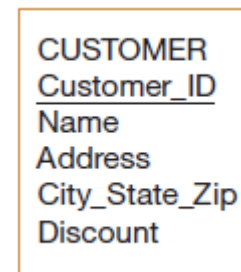
Each regular entity type in an E-R diagram is transformed into a relation. The identifier of the entity type becomes the primary key of the corresponding relation.

Each nonkey attribute of the entity type becomes a nonkey attribute of the relation. You should check to make sure that the primary key satisfies the following two properties:

1. The value of the key must uniquely identify every row in the relation.
2. The key should be nonredundant; that is, no attribute in the key can be deleted without destroying its unique identification.

For example :

CUSTOMER(Customer_ID,Name,Address,City_State_ZIP,Discount)



Setup in MS Access

- Create these tables first in Access Table Design View:

- **Table 1: STUDENT**

- **Student_ID** (AutoNumber, Primary Key)
- **First_Name** (Short Text, 25)
- **Last_Name** (Short Text, 25)
- **Email** (Short Text, 50)
- **Phone** (Short Text, 15)
- **Enrollment_Date** (Date/Time)

- **Table 2: COURSE**

- **Course_ID** (Short Text, 10, Primary Key)
- **Course_Name** (Short Text, 50)
- **Credits** (Number, Integer)
- **Department** (Short Text, 30)
- **Description** (Long Text)

- **Table 3: INSTRUCTOR**

- **Instructor_ID** (AutoNumber, Primary Key)
- **First_Name** (Short Text, 25)
- **Last_Name** (Short Text, 25)
- **Office** (Short Text, 20)
- **Hire_Date** (Date/Time)
- **Salary** (Currency)
- **Dept_Code** (Short Text, 10)

- **Table 4: DEPARTMENT**

- **Dept_Code** (Short Text, 10, Primary Key)
- **Dept_Name** (Short Text, 50)
- **Building** (Short Text, 30)
- **Budget** (Currency)

- **Table 5: OFFICE**

- **Office_Number** (Short Text, 10, Primary Key)
- **Building** (Short Text, 30)
- **Square_Feet** (Number, Integer)
- **Instructor_ID** (Number, Long Integer) ← Foreign Key referencing INSTRUCTOR

2. Represent Relationships

The procedure for representing relationships depends on both the degree of the relationship—unary, binary, ternary—and the cardinalities of the relationship.

Binary 1: N: A binary one-to-many (1:N) relationship in an E-R diagram is represented by adding the primary key attribute (or attributes) of the entity on the **one** side of the relationship as a foreign key in the relation that is on the **many** side of the relationship.

ORDER

<u>Order_Number</u>	Order_Date	Promised_Date	<u>Customer_ID</u>
57194	3/15/1X	3/28/1X	6390
63725	3/17/1X	4/01/1X	1273
80149	3/14/1X	3/24/1X	6390

Binary or unary one-to-one (1:1) relationship: between two entities A and B (for a unary relationship, A and B would be the same entity type), the relationship can be represented by any of the following choices:

1. Adding the primary key of A as a foreign key of B
2. Adding the primary key of B as a foreign key of A
3. Both of the above

2. Represent Relationships

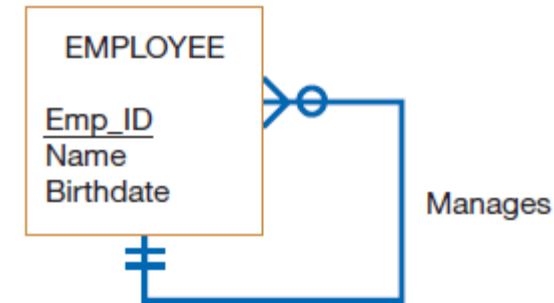
Binary and Higher-Degree $M:N$ Relationships: Suppose that there is a binary many-to-many ($M:N$) relationship (or associative entity) between two entity types A and B. For such a relationship, we create a separate relation C. The primary key of this relation is a composite key consisting of the primary key for each of the two entities in the relationship. Any nonkey attributes associated with the $M:N$ relationship are included with relation C.



2. Represent Relationships

Unary Relationships : a unary relationship is a relationship between the instances of a single entity type, which are also called *recursive relationships*.

one-to-many relationship named *Manages* that associates employees with another employee who is their manager.



For a unary 1:*N* relationship, the entity type (such as EMPLOYEE) is modeled as a relation.

The primary key of that relation is the same as for the entity type.

Then a foreign key is added to the relation that references the primary key values. A **recursive foreign key** is a foreign key in a relation that references the primary key values of that same relation:

EMPLOYEE(*Emp_ID*,*Name*,*Birthdate*,*#Manager_ID*)

In MS Access

- **Example A: 1:N Relationship - Department to Instructor**
 - **Rule:** Add primary key from "one" side as foreign key in "many" side
- **Example B: 1:1 Relationship - Instructor to Office Assignment**
- **Example C: M:N Relationship - Student to Course (Enrollment)**
 - **Create junction table:**
 - **Table 6: ENROLLMENT**
 - **Student_ID** (Number, Long Integer) ← Foreign Key
 - **Course_ID** (Short Text, 10) ← Foreign Key
 - **Semester** (Short Text, 10)
 - **Year** (Number, Integer)
 - **Grade** (Short Text, 2)
 - **Primary Key:** Composite of Student_ID + Course_ID + Semester + Year
- **Example D: Unary Relationship - Employee Management**
- **Create new table:**
- **Table 7: EMPLOYEE**
 - **Emp_ID** (AutoNumber, Primary Key)
 - **First_Name** (Short Text, 25)
 - **Last_Name** (Short Text, 25)
 - **Position** (Short Text, 30)
 - **Manager_ID** (Number, Long Integer) ← Recursive Foreign Key referencing Emp_ID

2. Represent Relationships

E-R Structure	Relational Representation
Regular entity	Create a relation with primary key and nonkey attributes.
Weak entity	Create a relation with a composite primary key (which includes the primary key of the entity on which this weak entity depends) and nonkey attributes.
Binary or unary 1:1 relationship	Place the primary key of either entity in the relation for the other entity or do this for both entities.
Binary 1:N relationship	Place the primary key of the entity on the one side of the relationship as a foreign key in the relation for the entity on the many side.
Binary or unary M:N relationship or associative entity	Create a relation with a composite primary key using the primary keys of the related entities, plus any nonkey attributes associative entity of the relationship or associative entity.
Binary or unary M:N relationship or associative entity with additional key(s)	Create a relation with a composite primary key using the primary keys of the related entities and additional primary key attributes associated with the relationship or associative entity, plus any nonkey attributes of the relationship or associative entity.
Binary or unary M:N relationship or associative entity with its own key	Create a relation with the primary key associated with the relationship or associative entity, plus any nonkey attributes of the relationship or associative entity and the primary keys of the related entities (as foreign key attributes).
Supertype/subtype	Create a relation for the superclass, which contains the primary relationship key and all nonkey attributes in common with all subclasses, plus create a separate relation for each subclass with the same primary key (with the same or local name) but with only the nonkey attributes related to that subclass.

Test with Sample Data

- Add all the data from the sample data file
 - Use “paste append”

3. *Merging Relations*

As part of the logical database design, normalized relations likely have been created from a number of separate E/R diagrams. Some of the relations may be redundant—they may refer to the same entities. If so, *you should merge those relations to remove the redundancy.*

This section describes merging relations, or view integration, which is the last step in logical database design and prior to physical file and database design.

an example of Merging Relations

Suppose that transforming an E-R diagram results in the following 3NF relation:

EMPLOYEE1(Emp_ID,Name,Address,Phone)

Modeling a second user interface might result in the following relation:

EMPLOYEE2(Emp_ID,Name,Address,Jobcode,Number_of_Years)

Because these two relations have the same primary key (Emp_ID) and describe the same entity, they should be merged into one relation. The result of merging the relations is the following relation:

EMPLOYEE(Emp_ID,Name,Address,Phone,Jobcode,Number_of_Years)

Notice that an attribute that appears in both relations (such as Name in this example) appears only once in the merged relation.

3. *Merging Relations*

When integrating relations, you must understand the meaning of the data and be prepared to resolve any problems that may arise in the process. Here, we describe and illustrate four problems that arise in view integration: synonyms, homonyms, dependencies between nonkeys, and class/subclass relationships:

1-Synonyms In some situations, two or more attributes may have different names but the same meaning, as when they describe the same characteristic of an entity.

Such attributes are called **synonyms**.

3. *Merging Relations*

For example, Emp_ID and Employee_Number may be synonyms.

When merging relations that contain synonyms, you have to define a single standardized name for the attribute and eliminate the other synonym.

Another alternative is to choose a third name to replace the synonyms.

For example, consider the following relations:

STUDENT1(Student_ID,Name)



STUDENT(SSN,Name,Address)

STUDENT2(Matriculation_Number,Name,Address)

3. *Merging Relations*

2-Homonyms In other situations, a single attribute name, called a **homonym**, may have more than one meaning or describe more than one characteristic.

For example, the term *account* might refer to a bank's checking account, savings account, loan account, or other type of account; therefore, *account* refers to different data, depending on how it is used. You should be on the lookout for homonyms when merging relations.

Consider the following example:

STUDENT1(Student_ID,Name,Address)

STUDENT2(Student_ID,Name,Phone_Number,Address)

In discussions with users, the systems analyst may discover that the attribute Address in STUDENT1 refers to a student's campus address, whereas in STUDENT2 the same attribute refers to a student's home address. To resolve this conflict, we would probably need to create new attribute names, and the merged relation would become

STUDENT(Student_ID,Name,Phone_Number,Campus_Address,Permanent_Address)

3. *Merging Relations*

3-Dependencies between Nonkeys

When two 3NF relations are merged to form a single relation, dependencies between nonkeys may result. For example, consider the following two relations:

STUDENT1(Student_ID, Major)

STUDENT2(Student_ID, Adviser)

Because STUDENT1 and STUDENT2 have the same primary key, the two relations may be merged:

STUDENT(Student_ID, Major, Adviser)

However, suppose that each major has exactly one adviser. In this case, Adviser is functionally dependent on Major.

If this dependency exists, then STUDENT is in 2NF but not 3NF because it contains a functional dependency between nonkeys. The analyst can create 3NF relations by creating two relations with Major as a foreign key in STUDENT:

STUDENT(Student_ID, #Major)

MAJOR ADVISER(Major, Adviser)

3. *Merging Relations*

4-Class/Subclass Class/subclass relationships may be hidden in user views or relations.

Suppose that we have the following two hospital relations:

PATIENT1(Patient_ID,Name,Address,Date_Treated)

PATIENT2(Patient_ID,Room_Number)

Initially, it appears that these two relations can be merged into a single PATIENT relation. However, suppose that there are two different types of patients: inpatients and outpatients.

PATIENT1 actually contains attributes common to all patients. PATIENT2 contains an attribute (Room_Number) that is a characteristic only of inpatients.

In this situation, you should create *class/subclass* relationships (1:1) for these entities:

PATIENT(Patient_ID,Name,Address)

INPATIENT(Patient_ID,Room_Number)

OUTPATIENT(Patient_ID,Date_Treated)

Let's Examine Normalization in MS Access

- Create the denormalized table with this data

Instructor_ID	First_Name	Last_Name	Office	Hire_Date	Salary	Dept_Code	Dept_Name	Building	Budget
1	John	Doe	T101	1/15/2020	\$75,000	CS	Computer Science	Tech Building	\$500,000
2	Jane	Smith	S205	8/20/2019	\$70,000	MATH	Mathematics	Science Hall	\$300,000
5	David	Wilson	T102	6/15/2020	\$72,000	CS	Computer Science	Tech Building	\$500,000

- Using MS Access Table Analyzer, normalize this table